МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
КАФЕДРА ИНТЕЛЛЕКТУАЛЬНЫХ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ

по лабораторной работе №4

Выполнил:
студент группы ПО-12
Снарский П.П.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Вариант 12

Ход работы

Общее задание:

Используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

Задание 1.

Напишите Python-скрипт, который анализирует активность разработчиков в указанном публичном GitHub-репозитории и составляет рейтинг самых активных контрибьюторов.

1. Запрашивает у пользователя имя репозитория (например, pallets/flask или microsoft/vscode).

2. Использует GitHub API для получения списка всех контрибьюторов.

3. Для каждого контрибьютора собирает данные:

- Количество коммитов
- Количество открытых pull requests
- Количество закрытых pull requests
- Количество открытых issues
- Количество закрытых issues
- Дата последней активности

4. Определяет самых активных разработчиков по суммарному вкладу.

5. Строит график вклада разработчиков (matplotlib / seaborn).

Код программы:

```
"""Devs activity in open-source project"""

from collections import defaultdict
import requests
import matplotlib.pyplot as plt

TOKEN = (
    "github_pat_11AOUI3YQ0IMsTkos8r0cb_AJN3yBQsIoRPiRdr4AEv"
    "uS7rg1wwtjPvb1dEi5PSsZIU57KQI43fk8nYgBD"
)
HEADERS = {"Authorization": f"token {TOKEN}"} if TOKEN else {}


def get_contributors(owner, repo):
    """Gets contributors for a repo"""
    url = f"https://api.github.com/repos/{owner}/{repo}/contributors"
```

```
    response = requests.get(url, headers=HEADERS, timeout=30)
    return response.json()


def get_user_stats(owner, repo, username):
    """Gets user stats for a repo"""
    stats = defaultdict(int)

    issues_url = f"https://api.github.com/repos/{owner}/{repo}/issues"
    params = {"creator": username, "state": "all", "per_page": 100}
    issues = requests.get(issues_url, headers=HEADERS, params=params,
timeout=30).json()

    for item in issues:
        if "pull_request" in item:
            if item["state"] == "open":
                stats["open_pr"] += 1
            else:
                stats["closed_pr"] += 1
        else:
            if item["state"] == "open":
                stats["open_issues"] += 1
            else:
                stats["closed_issues"] += 1

    commits_url = f"https://api.github.com/repos/{owner}/{repo}/commits"
    params = {"author": username, "per_page": 1}
    commits = requests.get(
        commits_url, headers=HEADERS, params=params, timeout=30
    ).json()

    if isinstance(commits, list) and commits:
        stats["last_activity"] = commits[0]["commit"]["author"]["date"]
    else:
        stats["last_activity"] = "N/A"

    return stats


def main():
    """Main function"""
    repo_input = input("Введите репозиторий (owner/repo): ")
    owner, repo = repo_input.split("/")

    print(f'Анализируем вклад контрибьюторов в "{repo_input}"...')

    contributors = get_contributors(owner, repo)

    results = []

    for user in contributors[:10]:
        username = user["login"]
        commits = user["contributions"]

        stats = get_user_stats(owner, repo, username)

        total_score = (
            commits
            + stats["open_pr"]
            + stats["closed_pr"]
            + stats["open_issues"]
            + stats["closed_issues"]
        )

        results.append(
            {
                "user": username,
```

```
                "commits": commits,
                "open_pr": stats["open_pr"],
                "closed_pr": stats["closed_pr"],
                "open_issues": stats["open_issues"],
                "closed_issues": stats["closed_issues"],
                "last_activity": stats["last_activity"],
                "score": total_score,
            }
        )

    results.sort(key=lambda x: x["score"], reverse=True)

    print("\nТОП-5 самых активных разработчиков:")
    for i, user in enumerate(results[:5], start=1):
        print(
            f'{i}. {user["user"]} - '
            f'{user["commits"]} коммитов, '
            f'{user["open_pr"] + user["closed_pr"]} PR, '
            f'{user["open_issues"] + user["closed_issues"]} issues'
        )

    names = [u["user"] for u in results[:5]]
    scores = [u["score"] for u in results[:5]]

    plt.figure()
    plt.bar(names, scores)
    plt.title(f"Top contributors: {repo_input}")
    plt.xlabel("Developers")
    plt.ylabel("Score")
    plt.xticks(rotation=30)

    filename = f"{repo}_contributors.png"
    plt.savefig(filename)

    print(f'\nГрафик сохранен в "{filename}"')


if __name__ == "__main__":
    main()
```

Результат работы:

```
Введите репозиторий (owner/repo): microsoft/vscode
Анализируем вклад контрибьюторов в "microsoft/vscode"...

ТОП-5 самых активных разработчиков:
1. bpasero - 14686 коммитов, 84 PR, 16 issues
2. jrieken - 12629 коммитов, 50 PR, 50 issues
3. Tyriar - 12495 коммитов, 26 PR, 74 issues
4. mjbvz - 10156 коммитов, 72 PR, 28 issues
5. joaomoreno - 8763 коммитов, 38 PR, 62 issues

График сохранен в "vscode_contributors.png"
```

Вывод: в ходе выполнения лабораторной работы научился работать с Github API, приобрел практические навыки написания программ для работы с REST API или GraphQL API